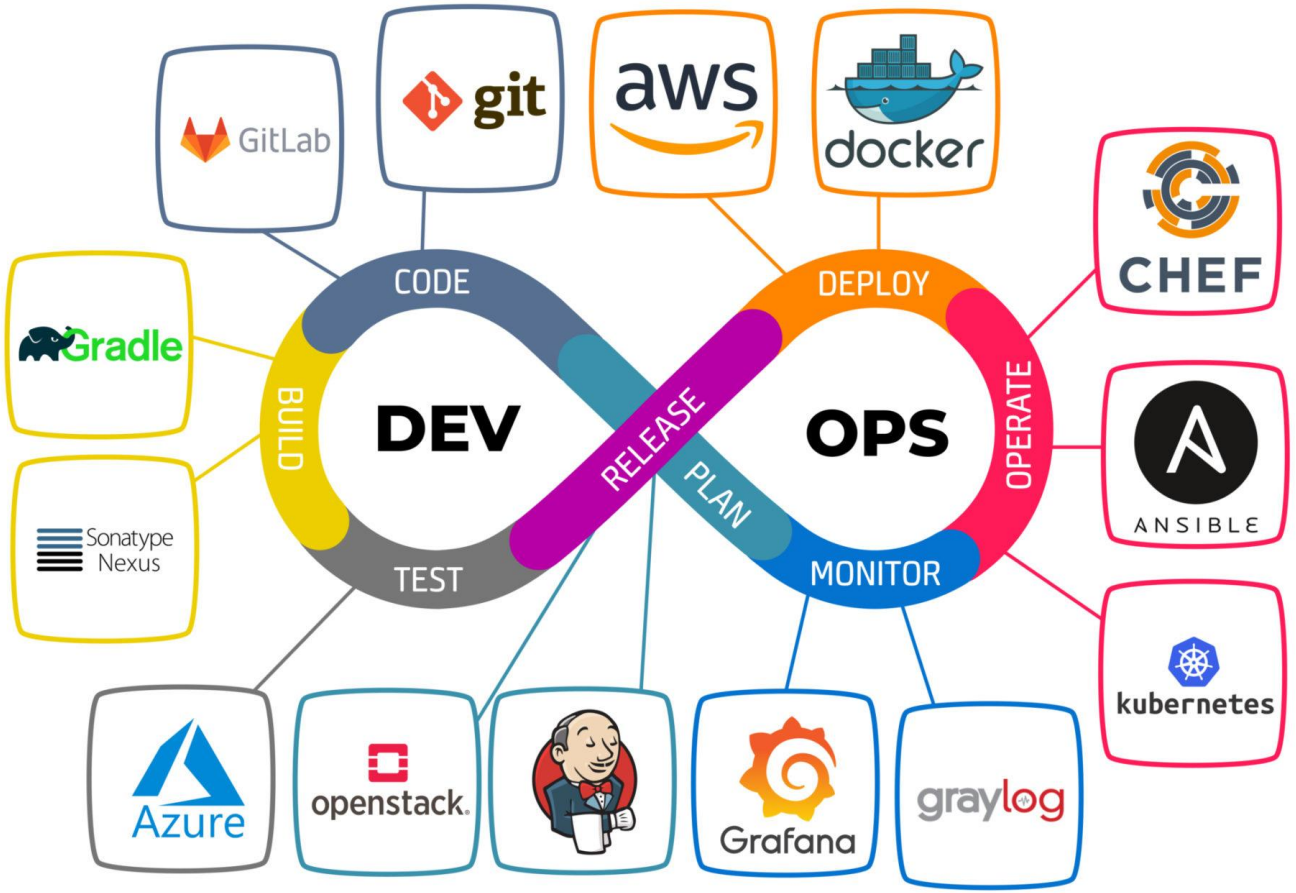
Transformation DevSecOps

Philosophie, pratiques et actualités – édition octobre 2026

Rappel DevOps



Rappel DevOps

Le « pourquoi » : silos Dev/Ops, déploiements longs et risqués, bugs découverts en production, « ça marche sur ma machine »

Philosophie CALMS : Culture, Automatisation, Lean (flux, petits lots), Mesure, Partage

Le cycle de vie : Plan → Code → Build → Test → Release → Deploy → Operate → Monitor, en boucle continue

Les « Three Ways » (The DevOps Handbook) : le flux, le feedback rapide, l'apprentissage continu

Les variantes du DevOps

DevSecOps : la sécurité intégrée à chaque étape (« shift left », complété par le « shift right »)

Réf. : Red Hat – Shift left vs shift right

GitOps : Git comme source unique de vérité, déploiement par réconciliation

MLOps / LLMOps : industrialiser le cycle de vie des modèles de ML et d'IA générative

DataOps : fiabiliser et automatiser les pipelines de données

AIOps : l'IA au service des opérations

Platform Engineering : une plateforme interne en libre-service (« golden paths ») qui embarque CI/CD et sécurité par défaut

FinOps : piloter les coûts cloud au rythme des déploiements

Variante DevOps
MLOps

Le **MLOps** étend les principes de DevOps (intégration continue, livraison continue, surveillance) au cycle de vie du *Machine Learning*.

Son objectif principal est de fiabiliser et d'automatiser le déploiement, la gestion et la surveillance des modèles de ML en production

Cycle de Vie et Différences Clés avec DevOps

Le cycle de vie MLOps est plus complexe que le DevOps classique car il ajoute une boucle supplémentaire : la gestion des **données** et du **modèle**.

Variante DevOps
MLOps

Phase MLOps	Description	Point de Focalisation
Data Engineering	Préparation, validation et gestion des données d'entraînement.	Assurer la qualité et l'intégrité des données.
Modelling (CI)	Entraînement, test et évaluation du modèle ML.	Versioning du code *et* du modèle (artefact) *et* des données.
Deployment (CD)	Déploiement du modèle sous forme d'API ou de service (A/B testing, Canary).	Gérer la transition du modèle vers la production.
Monitoring (CT - Continuous Training)	Surveillance de la performance du modèle en production.	Détection de la **dérive du modèle (Model Drift)** et des biais.

Variante DevOps
MLOps

Enjeux majeurs

- **Versionner le modèle et les données** : un modèle est un artefact binaire qui dépend du code et du jeu de données utilisé pour l'entraîner.
- **Reproductibilité** : pouvoir recréer exactement le même modèle en cas de problème.
- **Dérive (drift)** : le modèle perd en précision si les données réelles s'éloignent des données d'entraînement ; le MLOps met en place le réentraînement automatique.
- **Gouvernance** : traçabilité des modèles (registre, model cards) et exigences de l'AI Act européen pour les systèmes à haut risque.

Variante DevOps
Sécuriser le MLOps et les LLM

Les modèles et les applications d'IA générative ajoutent de nouvelles surfaces d'attaque :

- **Supply chain des modèles** : modèles téléchargés potentiellement piégés ; préférer le format safetensors au pickle, signer les modèles, tenir un AI-BOM (CycloneDX).
- **Empoisonnement** des données d'entraînement ou des bases documentaires RAG.
- **Injection de prompt**, directe ou indirecte (via un document, une page web, un ticket).
- **Agents trop autonomes** (« excessive agency ») : un agent IA avec trop de droits sur les outils, le code ou la production.
- **Fuite de données** sensibles et du prompt système.

Référence : OWASP Top 10 for LLM Applications 2025, avec l'injection de prompt en tête.

Variante DevOps
GitOps

Le **GitOps** est une méthodologie opérationnelle qui utilise **Git comme source unique de vérité** (Single Source of Truth) pour décrire l'état désiré des systèmes d'infrastructure et des applications.

Il applique les meilleures pratiques des développeurs (PR, revue de code, versioning) à l'infrastructure et au déploiement.

Variante DevOps
GitOps

Principes Fondamentaux

- **Infrastructure as Code (IaC) Déclarative** : L'intégralité de l'état du système (ex: manifestes Kubernetes, configurations Terraform) est stockée dans Git.
- **État Désiré dans Git** : Git contient la configuration cible souhaitée.
- **Agents de Synchronisation** : Un agent (ou "réconciliateur", ex: ArgoCD, Flux) est déployé dans l'environnement cible (ex: cluster Kubernetes).
- **Boucle de Réconciliation** : L'agent surveille en permanence la différence entre l'**état désiré** (dans Git) et l'**état actuel** du cluster. Si une différence est détectée, il déploie ou ajuste l'infrastructure pour que l'état actuel corresponde à Git.

Variante DevOps
GitOps

Avantages

- **Auditabilité** : chaque changement, même opérationnel, passe par une pull request ; l'historique est complet et immuable.
- **Sécurité** : modèle « pull », l'agent dans le cluster tire les changements ; la CI n'a plus besoin des identifiants de production et les accès humains SSH/console diminuent.
- **Déploiements plus rapides** : déployer devient un simple merge.
- **Outils de référence** : Argo CD et Flux (projets CNCF « graduated »), à sécuriser eux-mêmes : RBAC, commits signés, protection des branches.

Variante DevOps
DataOps

Objectif et Définition

Le **DataOps** est une approche collaborative, basée sur les processus et automatisée, visant à améliorer la qualité et à réduire le temps de cycle de la livraison des données (pipelines de données).

C'est l'application des principes DevOps à l'analyse de données.

Variante DevOps DataOps

Phase DataOps	Focalisation
Intégration et Qualité	Automatiser les tests de qualité des données dès l'ingestion (ex: schémas, valeurs manquantes).
Développement	Utiliser des environnements de test et de staging pour les pipelines de données avant de les exposer.
Livraison	Déploiement rapide des nouvelles sources ou transformations de données.
Surveillance	Suivi du débit, de la latence, et de la qualité (fraîcheur, précision) des données en temps réel.

Variante DevOps
DataOps

Le DataOps est le fondement du MLOps : un bon MLOps nécessite de bonnes pratiques de DataOps.

Le DataOps fournit les pipelines stables et de haute qualité que le MLOps utilise pour entraîner et valider ses modèles.

Variante DevOps
AIOps

Artificial Intelligence for IT Operations

L'**AIOps** utilise l'intelligence artificielle (IA) et le *Machine Learning* pour automatiser et améliorer les opérations informatiques (IT Operations).

Il s'agit de gérer les systèmes complexes à grande échelle en s'appuyant sur les données de monitoring.

Variante DevOps
AIOps

L'AIOps est généralement mis en œuvre en deux couches :

- **Big Data Layer** : Agrégation de toutes les données d'opérations (logs, métriques, événements, traces) provenant de toutes les sources de l'entreprise.
- **ML Layer** : Des algorithmes de *Machine Learning* sont appliqués à ces données pour :
  - **Réduction du Bruit** : Regrouper des milliers d'alertes en un seul incident significatif.
  - **Corrélation des Événements** : Identifier les liens de cause à effet entre des événements disparates.
  - **Détection d'Anomalies** : Identifier des comportements systèmes inhabituels avant qu'ils ne deviennent des pannes.
  - **Prédiction** : Prédire les pannes futures (Maintenance Prédictive).

Variante DevOps
AIOps

Impact sur les opérations

L'AIOps permet de passer d'une gestion réactive (réagir à une alerte) à une gestion proactive (prédire et prévenir l'incident), ce qui améliore le délai de rétablissement et la disponibilité des services.

En 2026, l'AIOps évolue vers des agents (« AI SRE ») capables de diagnostiquer et de proposer, voire d'appliquer, une remédiation : garder un humain dans la boucle pour les actions à risque, et tracer ce que fait l'agent.

Pourquoi le DevSecOps

Pourquoi maintenant ?

La chaîne d'approvisionnement logicielle est devenue une cible privilégiée :

- **xz-utils (2024)** : porte dérobée glissée pendant près de deux ans par un faux mainteneur dans une bibliothèque présente sur la plupart des distributions Linux, découverte de justesse.
- **tj-actions/changed-files (2025)** : une GitHub Action très utilisée, compromise, exposait les secrets CI des dépôts qui l'appelaient.
- **Shai-Hulud (2025)** : ver npm qui volait les jetons des développeurs pour publier des versions piégées d'autres paquets.
- **Code généré par IA** : volume de code en forte hausse, revu moins attentivement ; paquets « hallucinés » enregistrés par des attaquants (slopsquatting).
- **Réglementation** : CRA, NIS2 et DORA (UE) rendent la sécurité logicielle opposable.

OWASP Top 10 – édition 2025

Rang	Catégorie (nouveautés par rapport à 2021)
A01	Contrôle d'accès défaillant (absorbe désormais le SSRF)
A02	Mauvaise configuration de sécurité (remonte de la 5e place)
A03	Défaillances de la chaîne d'approvisionnement logicielle (nouveau, élargit « composants vulnérables »)
A04	Défaillances cryptographiques
A05	Injection
A06	Conception non sécurisée
A07	Défaillances d'authentification
A08	Défaillances d'intégrité des logiciels ou des données
A09	Défaillances de journalisation et d'alerte
A10	Mauvaise gestion des conditions exceptionnelles (nouveau)

Le cadre réglementaire européen

Texte	Qui / quoi	Échéances
Cyber Resilience Act (CRA)	Fabricants de produits comportant du logiciel : sécurité dès la conception, gestion des vulnérabilités, SBOM, mises à jour pendant la durée de support	Signalement des vulnérabilités activement exploitées et incidents graves (alerte 24 h, notification 72 h) depuis le 11 sept. 2026 ; application complète le 11 déc. 2027
NIS2	Entités essentielles et importantes de 18 secteurs : gestion des risques, sécurité de la chaîne d'approvisionnement, notification des incidents	Transposition due en oct. 2024 ; en France, le projet de loi « Résilience » n'est pas encore adopté (examen reporté en oct. 2026)
DORA (règlement UE 2022/2554)	Secteur financier et ses prestataires TIC : résilience opérationnelle, tests, risque tiers	Applicable depuis le 17 janv. 2025 – à ne pas confondre avec les métriques DORA
AI Act	Fournisseurs et déployeurs de systèmes d'IA, obligations selon le niveau de risque	Entrée en application progressive 2025-2027

DevSec ou SecDev ?

La sécurité occupe une place de plus en plus prépondérante dans les projets, et selon l'approche DevSecOps ou SecDevOps, son intégration varie significativement. Dans une démarche SecDevOps, la sécurité guide le projet en imposant un ensemble de règles, qu'elles soient juridiques ou spécifiques à une entreprise, plaçant ainsi la sécurité au cœur du développement de logiciel ou d'application.

L'approche DevSecOps se définit par l'intégration de la sécurité à chaque phase du cycle de développement, en partant d'une initiative originale de l'équipe de développement et en y intégrant progressivement la sécurité tout au long du processus.

DevSec ou SecDev ?

DevOps (Development + Operations) : Un mouvement visant à unir le développement de logiciels (Dev) et les opérations informatiques (Ops) pour raccourcir le cycle de vie du développement système et fournir une livraison continue de haute qualité .

DevSecOps (Development + Security + Operations) : C'est l'extension naturelle de DevOps, où la Sécurité (Sec) est intégrée dès le début et tout au long du cycle de vie ("Security as Code"). L'objectif est de rendre la sécurité transparente et automatisée, en évitant qu'elle ne soit un goulot d'étranglement de dernière minute.

> Le mot-clé est **"Shift Left"** : la sécurité est déplacée vers la gauche, c'est-à-dire le plus tôt possible dans le pipeline, dès la phase de conception et de codage.

DevSec ou SecDev ?

SecDevOps (Security + Development + Operations) : Bien que parfois utilisé, ce terme est moins courant et peut prêter à confusion.

Il pourrait signifier deux choses :

- Une simple inversion des mots pour désigner la même approche que DevSecOps.
- Une approche où la Sécurité est le point de départ ou le moteur principal de l'initiative, ce qui pourrait potentiellement entraîner une focalisation excessive sur les outils de sécurité au détriment de la vitesse et de la collaboration. Dans le cadre DevSecOps, la sécurité est un partenaire intégré, pas un chef d'orchestre distinct.

DevSec ou SecDev ?

Dans le domaine de l'intégration de la sécurité au sein du processus de développement, **l'interaction entre les équipes** joue un rôle crucial.

Le modèle DevSecOps promeut une collaboration équilibrée entre développeurs, opérationnels et experts en sécurité.

Le modèle SecDevOps accorde quant à lui une prédominance à l'équipe de sécurité dans le processus de développement.

Secure by Design

Le DevSecOps s’enracine dans la **culture collaborative du DevOps**, étendue à la sécurité.

Il s’agit de commencer les échanges dès les premières phases du projet, établissant les normes de sécurité et de gouvernance pour anticiper les besoins et les problématiques de sécurité dès le démarrage.

Cela permet de concevoir des solutions partagées, et même des stratégies de contournement si nécessaire, en tenant compte des contraintes budgétaires qui pourraient influencer le coût global du projet.

Secure by Design

Il est essentiel d’adapter le niveau de sécurité aux enjeux business du projet. Pour les initiatives critiques et réglementées, le SecDevOps pourrait être plus pertinent, privilégiant la sécurité dès le départ.

En revanche, pour les projets avec des risques sécuritaires moins élevés, la pratique du DevSecOps pourrait être privilégiée, intégrant la sécurité de manière plus fluide et adaptable à la progression du projet.

Références : engagement « Secure by Design » de la CISA (2024), guides de l'ANSSI, et désormais les exigences essentielles du CRA.

Les 5 piliers du DevSecOps

Les 5 piliers

L’approche DevSecOps va répondre à cinq piliers qui permettent de répondre aux exigences de la sécurité dans le cycle DevOps :

- **Réduction de la vulnérabilité et des surfaces d’attaques** : L’objectif est de minimiser autant que possible les vulnérabilités dès le début du développement, afin d’éviter les failles de sécurité qui pourraient survenir après coup.
- **Accélération du time to market** : L’ambition est d’accélérer la livraison des projets. Avec DevSecOps, la sécurité est prise en compte dès le début du processus et non après, évitant ainsi les retours en arrière coûteux et chronophages.

Les 5 piliers

- **Collaboration accrue entre les équipes** : DevSecOps promeut une synergie entre les développeurs, les opérateurs de sécurité et les autres membres de l'équipe projet, favorisant ainsi un travail d'équipe fluide et efficace. Cela implique également un top management qui soutient et encourage ces pratiques.
- **Réponse aux exigences réglementaires** : Que ce soit pour répondre à des réglementations spécifiques à certains secteurs comme la santé, la défense ou la finance, ou à des règles propres à une entreprise, DevSecOps permet de répondre adéquatement aux exigences réglementaires variées, adaptées à chaque contexte.

Les 5 piliers

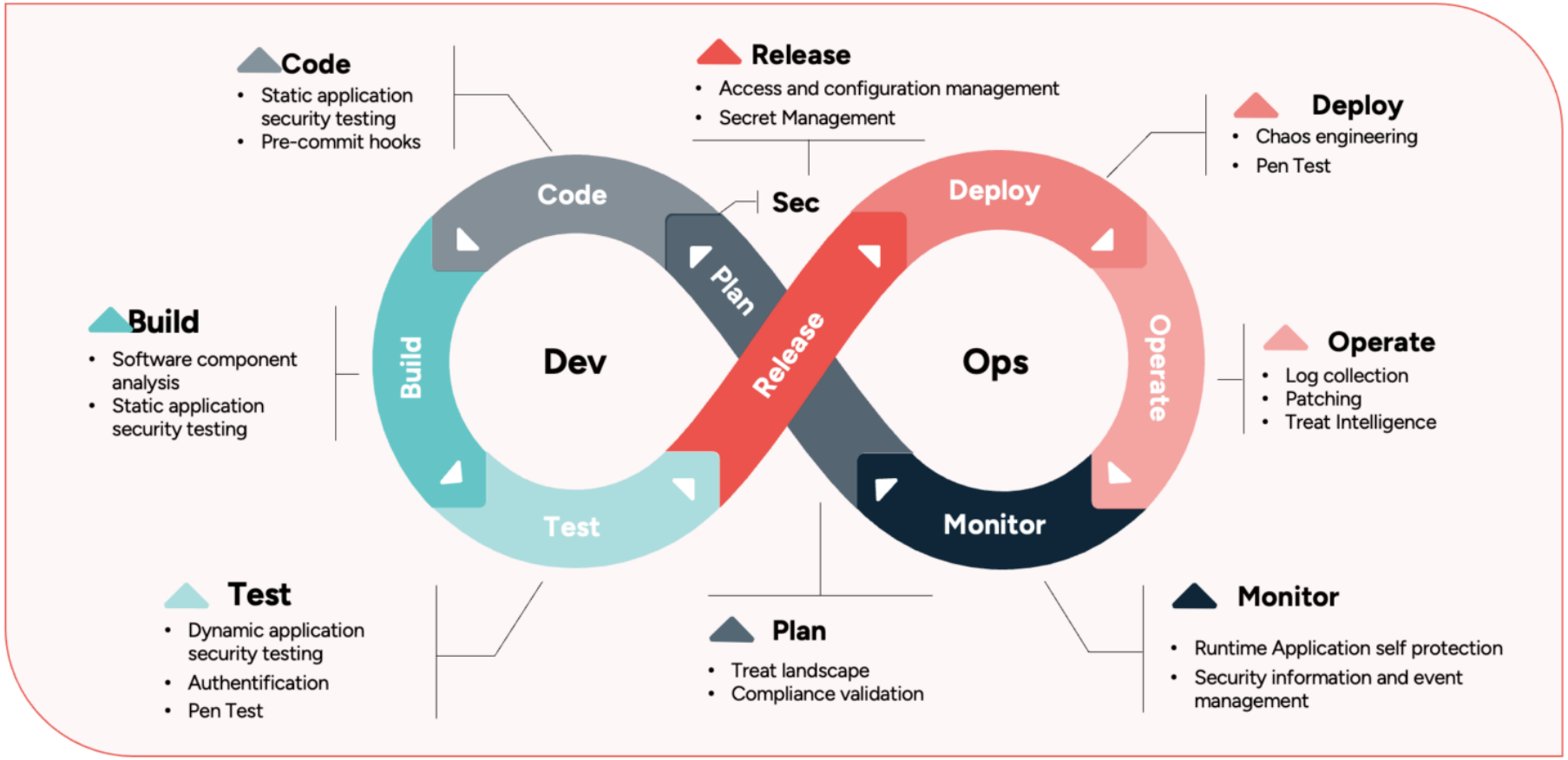
- **Confiance du client** : Au cœur de DevSecOps, il y a la volonté de renforcer la confiance des clients, souvent ébranlée par les risques de cyberattaques ou de pertes de données.

  Cela s'étend également aux exigences des compagnies d'assurance qui demandent désormais des garanties importantes concernant la sécurité des données, des accès…

En 2026, ces attentes deviennent des obligations : le CRA et NIS2 rendent la sécurité du logiciel et de sa chaîne d'approvisionnement opposable.

L’approche DevSecOps

Les grands principes du DevSecOps



Méthode

- **Définir un état des lieux** : dans l'élaboration d'une stratégie DevSecOps efficace, il est crucial de commencer par un audit de votre situation actuelle pour déterminer le niveau de maturité sur les différents aspects du cycle de déploiement CI/CD.

  Cela permet de savoir à quel niveau notre équipe sécurité est mature pour pouvoir s'intégrer aux équipes de développement et aux équipes opérationnelles.

- **Prioriser les chantiers** : cela permet de définir les chantiers en fonction des priorités stratégiques de l'entreprise, en mettant en avant les initiatives critiques qui génèrent le plus de valeur.

Méthode

- **Adopter une approche itérative** : les menaces de sécurité évoluent, tout comme les outils pour les contrer. S'enfermer dans un long projet de mise en conformité qui s'étendrait sur des mois, voire des années, pourrait vous laisser à la traîne, avec des solutions obsolètes avant même le déploiement.

- **Formation et sensibilisation** : il s'agit d'un processus continu. Les équipes doivent être constamment formées à collaborer et à intégrer la sécurité dans chaque phase du développement et de l'opérationnel.

- **Amélioration continue** : l'amélioration continue, testée et ajustée par les retours continus des utilisateurs et des clients, est fondamentale pour une adaptation fluide et efficace aux exigences évolutives. C'est en restant à l'écoute et en ajustant les pratiques que l'on parvient à un alignement parfait entre la sécurité, l'efficacité et les besoins des utilisateurs.

Référentiels de maturité

Référentiel	Usage
OWASP SAMM	Évaluer et piloter la maturité sécurité du développement (5 fonctions métier, 3 niveaux)
OWASP DSOMM	Maturité propre au DevSecOps : activités concrètes par niveau, utile pour bâtir une feuille de route
NIST SSDF (SP 800-218)	Pratiques de développement sécurisé, référence pour les fournisseurs du gouvernement américain
SLSA	Niveaux de sécurité de la chaîne de build (provenance, intégrité des artefacts)
OpenSSF Scorecard	Note automatique de la santé sécurité des dépôts open source, y compris les siens
EBIOS RM (ANSSI)	Méthode française d'analyse de risques, adaptée aux projets sensibles ou réglementés

Shift left… et shift right

Le shift left ne suffit pas : tout ne se voit pas avant la production.

- **Shift left** : sécurité dès la conception et dans l'IDE, contrôles en pre-commit et en pull request, modélisation des menaces au démarrage.
- **Shift right** : DAST sur l'application déployée, détection runtime, observabilité, chaos engineering, bug bounty.
- **Paved road / golden path** : l'équipe plateforme fournit des pipelines sécurisés par défaut ; le chemin sûr devient le plus simple.
- **Security champions** : un relais sécurité formé dans chaque équipe plutôt qu'une équipe sécurité goulot d'étranglement.
- **Prioriser par le risque** : exploitabilité (EPSS, catalogue KEV de la CISA) et atteignabilité du code vulnérable, pas seulement le score CVSS.

Build once, Scan, Promote (BSP)

Principe : on construit l'artefact **une seule fois**, on le contrôle, puis on **promeut ce même artefact** d'environnement en environnement, sans jamais le reconstruire.

- **Build once** : une image immuable, identifiée par son digest (sha256) et non par un tag modifiable comme « latest ».
- **Scan** : SCA, scan d'image, SBOM et signature au moment du build ; rescan continu dans le registre.
- **Promote** : dev → recette → préprod → prod en déplaçant la référence (digest, tag de promotion), après des quality gates à chaque étape.
- **Configuration externalisée** : seuls la config et les secrets changent selon l'environnement (variables, Vault), jamais le binaire.
- **Pourquoi** : ce qui a été testé et scanné est exactement ce qui part en production ; traçabilité de bout en bout, rollback simple, provenance vérifiable (SLSA, Cosign).

L’analyse des risques

Deux éléments sont souvent négligés au démarrage d’un projet :

- **Le paysage des menaces** : en fonction de l’outil qui va être déployé, du service concerné, de sa zone géographique, etc. Dans le DevSecOps, différents facteurs doivent être étudiés dans le cadre de la définition du paysage des menaces.

  L’équipe sécurité, ou une équipe dédiée à l’analyse des risques, doit établir cette étude pour anticiper les menaces potentielles.

- **La Compliance Validation** qui englobe toutes les normes de sécurité nécessaires à la livraison du projet suivant les exigences internes et légales.

L’analyse des risques

- Il faut démarrer par une **analyse de risque approfondie** où tous les risques sont identifiés et priorisés. Une gouvernance prédéfinie par l’équipe de sécurité dictera les règles à suivre pour couvrir les menaces.

- Bien sûr, l’étendue de cette analyse de risque dépend de l’ampleur du projet. Un changement majeur à apporter à une solution nécessitera une réévaluation complète des risques, contrairement à des modifications mineures qui ne justifieraient pas une nouvelle analyse approfondie.

Méthodes de modélisation des menaces : STRIDE, LINDDUN (vie privée), outillées par OWASP Threat Dragon ; EBIOS RM (ANSSI) pour une analyse plus formelle.

L’analyse des risques

Une analyse de risque peut coûter cher, c’est pourquoi elle doit être pondérées par rapport aux moyens de l’entreprise.

Chaque projet mérite une évaluation du risque, mais il revient au chef de projet et aux équipes de définir la profondeur nécessaire de cette étude.

Enfin, rassembler les différentes parties prenantes pour une première évaluation des risques est une étape simple et peu coûteuse.

Avant de s’engager dans une analyse de risque plus élaborée et onéreuse, il est judicieux d’évaluer si les bénéfices justifieraient son coût et sa complexité ajoutée.

Les parties prenantes

Les rôles clefs

DevOps Engineer : automatisation, CI/CD, infrastructure

Platform Engineer : plateforme interne, golden paths, pipelines sécurisés par défaut

Security Engineer / AppSec : politique de sécurité, outillage, audits, modélisation des menaces

Security Champion : relais sécurité au sein de chaque équipe de développement

Developer : code, tests, revues de code, correction des vulnérabilités

QA Engineer : tests automatisés, tests de sécurité

SRE : disponibilité, performance, gestion des incidents

Product Owner : priorisation, y compris de la dette de sécurité

Architect : design, standards, patterns, secure by design

RSSI : gouvernance, conformité, gestion des risques

Les compétences

- Scripting (Python, Bash, PowerShell)
- Cloud (AWS / Azure / GCP) et sécurité cloud (IAM, CSPM)
- Conteneurs et orchestration (Docker, Kubernetes)
- CI/CD (GitLab CI, GitHub Actions, Jenkins) et sécurisation des pipelines
- IaC (Terraform / OpenTofu, Ansible) et policy as code (OPA, Kyverno)
- Sécurité applicative (OWASP Top 10 2025, ASVS, cryptographie, pentest)
- Supply chain (SBOM, signature, SLSA)
- Modélisation des menaces (STRIDE, EBIOS RM)
- Sécurité de l'IA (OWASP Top 10 LLM, usage sûr des assistants de code)

Les anti-patterns

DevSecOps, nouvel outil

Erreur : acheter des outils (scanners, plateforme CI) sans changement culturel

Conséquence : silos maintenus, automatisation partielle, alertes ignorées

Leçon : le DevSecOps est d'abord une culture, l'outil vient ensuite

Sécurité, goulot d’étranglement

Erreur : validation sécurité manuelle en fin de cycle

Conséquence : ralentissement, frustration, contournements

Leçon : automatiser et intégrer la sécurité tôt

Too much, Too fast

Erreur : transformation totale en 3 mois

Conséquence : surcharge cognitive, résistance au changement

Leçon : transformation progressive, équipe par équipe

Le mur d'alertes

Erreur : brancher tous les scanners en mode bloquant dès le premier jour

Conséquence : des milliers de faux positifs, fatigue des équipes, contournements (--no-verify, règles désactivées)

Leçon : démarrer en mode informatif, ne bloquer que les nouvelles vulnérabilités critiques, centraliser et dédupliquer les résultats

L'IA, angle mort

Erreur : accepter le code généré par un assistant IA sans revue ni contrôle

Conséquence : vulnérabilités, secrets en dur, dépendances inventées par l'IA puis enregistrées par des attaquants

Leçon : même pipeline et même revue pour le code IA, une politique d'usage des assistants, vérification des dépendances

KPIs DevSecOps

Métriques DORA

Le mouvement DevOps naît en 2007-2009. Le rapport State of DevOps paraît à partir de 2013 avec Puppet, puis Nicole Forsgren, Jez Humble et Gene Kim fondent DORA (DevOps Research and Assessment) en 2015.

Rachetée par Google Cloud fin 2018, DORA publie toujours un rapport annuel ; celui de 2025 est consacré au développement logiciel assisté par l'IA.

Objectif : relier pratiques techniques et culture à la performance de livraison logicielle (livre de référence : Accelerate, 2018).

Métriques DORA

Les « quatre clés » historiques ont évolué : DORA suit désormais **cinq métriques** en deux familles.

- **Débit (throughput)**
  - Délai d'exécution des changements – **Lead time for changes**
  - Fréquence de déploiement – **Deployment frequency**
  - Délai de rétablissement – **Failed deployment recovery time** (ex-MTTR)
- **Instabilité (instability)**
  - Taux de défaillance des changements – **Change failure rate (CFR)**
  - Taux de retravail – **Rework rate**, nouveau

Elles se lisent ensemble : la vitesse sans la stabilité ne vaut rien.

Métriques DORA

Ces métriques permettent de situer une équipe par rapport à l'industrie (niveaux élite, élevé, moyen, faible) et surtout de suivre sa propre progression dans le temps.

Les derniers rapports insistent aussi sur le bien-être des équipes, la qualité de la plateforme interne et l'effet de l'IA, qui peut accélérer le débit tout en augmentant l'instabilité.

Attention : une métrique transformée en objectif individuel cesse d'être une bonne mesure (loi de Goodhart). On mesure des équipes et des systèmes, pas des personnes.

Niveaux de performance DORA

Métrique	Low	Medium	High	ELITE
Deployment frequency	Tous les 1 à 6 mois	Hebdo à mensuel	Quotidien à Hebdo	A la demande
Leat Time for Changes	1 à 6 mois	1 semaine à 1 mois	1 jour à 1 semaine	< 1 jour
Change Failure Rate	40%	10%	20%	5 %
Failed deployment recovery time	1 semaine à 1 mois	< 1 jour	< 1 jour	< 1 heure
Rework Rate	Pas de seuil publié par DORA			

Source : Accelerate State of DevOps 2024 (dernier rapport à publier ces niveaux ; le rapport 2025 ne les publie plus). Curiosité 2024 : le niveau Medium affiche un CFR plus bas que High, mais livre beaucoup moins souvent.

Fréquence des déploiements
Deployment frequency

La fréquence de déploiement (DF) est une mesure du rendement moyen.

Il s'agit de la fréquence moyenne à laquelle le code est déployé en production sur une période donnée. Cette métrique peut ainsi être utilisée pour évaluer la fréquence à laquelle une équipe d'ingénierie livre de la valeur aux clients.

Mesurer la fréquence de déploiement au fil du temps peut aider les équipes à identifier des moyens d'améliorer leur vitesse de livraison. DORA a notamment constaté que les équipes DevOps les plus performantes effectuent généralement des déploiements plus petits et plus fréquents.

Si une équipe est peu performante dans ce domaine, elle peut utiliser cette information pour identifier des moyens spécifiques de s'améliorer : par exemple, en décomposant le travail en lots plus petits ou en créant des pull requests moins importantes.

Délai d'exécution des changements
Lead time for changes

Le délai d'exécution des changements (LT) mesure le temps nécessaire pour qu'un changement de code arrive en production, **du commit au déploiement**.

On suit plutôt la médiane sur une période, plus robuste que la moyenne face aux valeurs extrêmes.

Un délai long révèle des goulots d'étranglement : revues qui traînent, tests lents, validations manuelles (souvent la sécurité en fin de cycle), mises en production groupées.

Leviers : petits lots, tests et contrôles de sécurité automatisés dans le pipeline, trunk-based development.

Délai de rétablissement
Failed deployment recovery time

Le délai de rétablissement après un déploiement raté (failed deployment recovery time, anciennement MTTR) mesure le temps nécessaire pour revenir à un état sain après un changement qui a provoqué une défaillance en production.

DORA l'a recentré sur les incidents causés par un déploiement : une panne d'origine externe (coupure d'un fournisseur, par exemple) n'entre pas dans le calcul.

Il suppose d'identifier vite la défaillance (observabilité), puis de corriger ou de revenir en arrière : rollback, feature flags, déploiements canary.

Délai de rétablissement
Failed deployment recovery time

Plus ce délai est court, mieux c'est.

Les équipes les plus performantes se rétablissent en moins d'une heure ; la plupart ont besoin de plusieurs heures, voire d'une journée.

Pour l'améliorer : un plan de réponse aux incidents clair et connu de tous, des déploiements réversibles, des runbooks et des post-mortems sans recherche de coupable.

Taux de défaillance des changements
Change failure rate

Le taux de défaillance des changements (CFR) indique le pourcentage de déploiements qui provoquent une défaillance en production nécessitant une remédiation (rollback, correctif, hotfix).

CFR = déploiements en échec / nombre total de déploiements

Quand on pousse les équipes à livrer vite, des anomalies échappent inévitablement à leur vigilance. Un CFR élevé signale un problème de qualité.

Selon le rapport State of DevOps 2021, la plupart des équipes se situent entre 0 et 15 %.

Taux de défaillance des changements
Change failure rate

Ainsi, le CFR sert de contrepoint critique aux métriques de fréquence de déploiement et de délai d'exécution des changements, qui se focalisent sur la vitesse.

Déployer fréquemment est une bonne chose, mais pas si cela implique de sacrifier constamment la qualité – et de devoir passer du temps à corriger les problèmes qui s'ensuivent.

En analysant le CFR, les responsables peuvent s'assurer que les équipes optimisent à la fois la stabilité et le rendement.

Taux de retravail
Rework rate

Cinquième métrique DORA : la part des déploiements non planifiés, réalisés pour corriger un problème en production.

Rework rate = déploiements non planifiés / nombre total de déploiements

Elle complète le CFR : une équipe peut afficher peu d'incidents tout en passant une part importante de son temps à « recoller les morceaux ».

Avec l'essor du code généré par IA, c'est un bon indicateur pour vérifier que le gain de vitesse ne se paie pas en corrections.

D'autres métriques

Métriques de sécurité

- Vulnérabilités ouvertes par sévérité, et surtout leur âge
- Délai moyen de correction par sévérité, comparé à un SLA (ex. critique < 7 jours)
- Part des vulnérabilités hors SLA
- Couverture : % de dépôts et de pipelines équipés des contrôles (SAST, SCA, secrets, IaC)
- Supply chain : % d'images signées et accompagnées d'un SBOM
- Secrets détectés : bloqués avant le push vs découverts après
- Taux de conformité aux politiques de sécurité (policy as code)

Métriques de qualité

- Couverture de tests (code coverage)
- Dette technique
- Code smells et duplication
- Respect de la quality gate (ex. SonarQube)
- Délai de revue des pull requests

Métriques culturelles

- Satisfaction des équipes (sondages, cadres SPACE ou DevEx)
- Charge cognitive et temps perdu en friction
- Temps d'onboarding
- Turnover
- % de développeurs formés à la sécurité, nombre de security champions